

Доклад

Режим электронной книги (ЕСВ)

Кадрова Ангелина Александровна

Содержание

1	Введение	4
2	Математическое определение режима ECB	6
2.1	Разбиение и дополнение (Padding)	6
2.2	Поблочное шифрование	6
2.3	Формирование шифртекста	7
2.4	Дешифрование	7
3	Реализация алгоритма на языке Julia	8
4	Криптографический анализ и уязвимости	11
4.1	Детерминированность	11
4.2	Отсутствие межблочной диффузии	11
4.3	Уязвимость к подмене блоков (Malleability)	12
4.4	Атака по словарю (Codebook Attack)	12
5	Наглядная демонстрация уязвимости: шифрование изображения пингвина Tux	13
6	Области применения режима	15
6.1	Шифрование случайных данных	15
6.2	Аппаратные реализации	15
6.3	Криптографические примитивы	15
7	Заключение	16
	Список литературы	17

Список иллюстраций

3.1	Результаты выполнения алгоритма	10
5.1	Шифрование пингвина Tux	14

1 Введение

Блочные шифры являются фундаментальным инструментом симметричной криптографии. Однако сам по себе блочный шифр способен обрабатывать только данные фиксированного размера. Для работы с сообщениями произвольной длины необходимы специальные алгоритмы, называемые режимами шифрования (modes of operation).

Цель доклада – изучить режим электронной кодовой книги.

Задачи:

- Рассмотреть математическое определение режима ECB
- Реализовать алгоритм на языке Julia
- Проанализировать криптографические уязвимости
- Определить области допустимого применения

Режим электронной кодовой книги (Electronic Codebook, ECB) является простейшим и исторически первым стандартизированным режимом шифрования. Его название происходит от аналогии с физическими кодовыми книгами, где слова или фразы заменялись на зафиксированные шифрованные эквиваленты. В режиме ECB каждый блок шифруется независимо от всех остальных блоков с использованием одного и того же ключа[1].

Исторически режим ECB был впервые формализован в 1980 году в стандарте FIPS 81, описывающем режимы работы блочного шифра DES, принятого Национальным бюро стандартов США (ныне NIST) в 1977 году. Долгое время он считался базовым способом шифрования данных и применялся

в первых реализациях криптографических систем. Однако по мере развития криптоанализа и роста требований к безопасности стали проявляться его фундаментальные недостатки, что привело к постепенному отказу от ЕСВ в пользу более совершенных режимов[2].

2 Математическое определение режима ЕСВ

Пусть задан блочный шифр E_k с размером блока n бит и ключом k . Процесс шифрования в режиме ЕСВ определяется следующим образом.

2.1 Разбиение и дополнение (Padding)

Исходное сообщение M разбивается на последовательность блоков длины n бит. Если длина сообщения не кратна n , применяется функция дополнения (padding), например, добавление пробелов:

$$M' = M_1 \| M_2 \| \dots \| M_m$$

где $M_i \in \{0, 1\}^n$, а $\|$ обозначает операцию конкатенации.

2.2 Поблочное шифрование

Функция шифра E_k применяется последовательно и независимо к каждому отдельному блоку открытого текста M_i :

$$C_i = E_k(M_i), \quad \text{для } i = 1, 2, \dots, m$$

2.3 Формирование шифртекста

Зашифрованный текст C формируется путём конкатенации зашифрованных блоков:

$$C = C_1 \| C_2 \| \dots \| C_m$$

2.4 Дешифрование

Выполняется независимо для каждого блока путём применения обратной функции D_k :

$$M_i = D_k(C_i), \quad \text{для } i = 1, 2, \dots, m$$

Ключевой особенностью ЕСВ является **полная независимость** обработки каждого блока[1].

3 Реализация алгоритма на языке Julia

Ниже представлена упрощенная реализация режима ECB. Код демонстрирует основную логику режима: разбиение на блоки, независимое шифрование каждого блока, сборку результата и дешифрование.

```
function crypt_block(block, key_chars)
    result = Char[]
    for (i, c) in enumerate(block)
        key_val = Int(key_chars[(i-1) % length(key_chars) + 1])
        shift = (key_val + i) % 256
        push!(result, Char(Int(c) ⊖ shift))
    end
    return result
end
```

```
function ecb_encrypt(text, key, block_size=8)
    println("Получен текст длиной $(length(text)) символов")
    padded = collect(text)
    original_len = length(padded)
    while length(padded) % block_size != 0
        push!(padded, ' ')
    end
end
```



```

println("Дополнено до $(length(padded)) символов")
println("Разбитие на блоки $block_size символов: ")
for i in 1:block_size:length(padded)
    block = padded[i:i+block_size-1]
    println("Блок $((i-1)÷block_size+1): '$(join(block))'")
end
key_chars = collect(key)
ciphertext = Char[]
for i in 1:block_size:length(padded)
    block = padded[i:i+block_size-1]
    encrypted_block = crypt_block(block, key_chars)
    append!(ciphertext, encrypted_block)
end
return ciphertext, original_len
end

function ecb_decrypt(ciphertext, key, original_len, block_size=8)
    key_chars = collect(key)
    plaintext = Char[]
    for i in 1:block_size:length(ciphertext)
        append!(plaintext, crypt_block(ciphertext[i:i+block_size-1], key_chars))
    end
    return join(plaintext[1:original_len])
end

function demonstrate_ecb()
    text = "Москва! Москва! Говорит Москва!"
    key = "ключ"
    println("Исходный текст: '$text'")

```

```

encrypted, _ = ecb_encrypt(text, key)
ascii_codes = [Int(c) for c in encrypted]

ascii_str = join(ascii_codes, " ")
println("\nПолный шифртекст:")
println(ascii_str)

println("\nЗашифрованные блоки:")
for i in 1:8:length(encrypted)
    block_str = join(ascii_codes[i:i+7], " ")
    println("Блок  $((i-1) \div 8 + 1)$ :  $block\_str$ ")
end
end
demonstrate_ecb()

```

После запуска получим следующее (рис. 3.1):

```

Исходный текст: 'Москва! Москва! Говорит Москва!'
Получен текст длиной 31 символов
Дополнено до 32 символов
Разбитие на блоки 8 символов:
Блок 1: 'Москва! '
Блок 2: 'Москва! '
Блок 3: 'Говорит '
Блок 4: 'Москва! '

Полный шифртекст:
1063 1027 1040 1137 1037 1137 116 111 1063 1027 1040 1137 1037 1137 116 111 1064 1027 1123 1141 1151 1145 1047 111 1063 1027 1040 1137 1037 1137 116 111

Зашифрованные блоки:
Блок 1: 1063 1027 1040 1137 1037 1137 116 111
Блок 2: 1063 1027 1040 1137 1037 1137 116 111
Блок 3: 1064 1027 1123 1141 1151 1145 1047 111
Блок 4: 1063 1027 1040 1137 1037 1137 116 111

Дешифрованный текст: 'Москва! Москва! Говорит Москва!'

```

Рис. 3.1: Результаты выполнения алгоритма

В ходе выполнения алгоритма исходное сообщение длиной 31 символ было дополнено до 32 символов (для кратности размеру блока $n = 8$) и разделено на четыре блока, три из которых содержали идентичную фразу «Москва!». Результат шифрования наглядно демонстрирует недостаток режима ЕСВ. Из одинаковых блоков текста получились абсолютно идентичные зашифрованные блоки.

4 Криптографический анализ и уязвимости

4.1 Детерминированность

$$M_i = M_j \Rightarrow C_i = C_j$$

Одинаковые блоки открытого текста всегда порождают одинаковые блоки зашифрованного текста. Это нарушает идею незаметности.

4.2 Отсутствие межблочной диффузии

В ECB изменение бита в блоке M_i влияет строго на соответствующий блок C_i и не распространяется на другие блоки. С одной стороны, это обеспечивает помехоустойчивость: повреждение одного зашифрованного блока при передаче не затрагивает остальные. С другой стороны, это означает отсутствие межблочной диффузии, свойства, при котором изменение одного блока открытого текста влияет на все последующие блоки шифртекста. Межблочная диффузия является одним из базовых требований к криптостойким режимам шифрования, поскольку она затрудняет локальный криптоанализ и подмену отдельных фрагментов данных. В ECB такое распространение отсутствует[1].

4.3 Уязвимость к подмене блоков (Malleability)

Поскольку блоки шифруются независимо, то злоумышленник может:

- Переставить блоки местами: $C_1 \| C_2 \| C_3 \rightarrow C_3 \| C_1 \| C_2$
- Дублировать блоки: $C_1 \| C_2 \rightarrow C_1 \| C_1 \| C_2$
- Вставить известные блоки из другого зашифрованного текста.

При дешифровании получаются осмысленные, но модифицированные данные. ЕСВ не обеспечивает целостности данных.

4.4 Атака по словарю (Codebook Attack)

Название режима «электронная кодовая книга» напрямую связано с атакой по словарю. Ее суть заключается в следующем: поскольку каждый блок открытого текста всегда отображается в один и тот же блок шифртекста, злоумышленник может составить таблицу соответствий «открытый блок $\rightarrow$ шифртекст» для наиболее вероятных фрагментов сообщения (типичных слов, стандартных заголовков, известных структур данных).

В дальнейшем, перехватив новый шифртекст, зашифрованный тем же ключом, атакующий может путём простого сопоставления с составленным словарём восстановить значительную часть исходного сообщения, **вообще не зная секретного ключа**. Фактически, шифр превращается в обычную кодовую книгу, набор фиксированных соответствий, которые можно заранее вычислить или подсмотреть.

Особенно опасна эта атака для сообщений с предсказуемой структурой: протоколов связи, баз данных, файловых форматов, где многие блоки заведомо известны или легко угадываются[3].

5 Наглядная демонстрация уязвимости: шифрование изображения пингвина Tux

Классической наглядной иллюстрацией (рис. 5.1) режима ECB является шифрование рисунка пингвина Tux (талисмана Linux) [4].



Рис. 5.1: Шифрование пингвина Тух

Исходное изображение содержит большие однородные области, например, сплошной белый фон и черные элементы тела пингвина. Когда картинка шифруется в режиме ECB, повторяющиеся блоки пикселей превращаются в одинаковые зашифрованные блоки.

В результате на зашифрованной картинке полностью сохраняются геометрические контуры и узнаваемый силуэт пингвина Тух[5].

6 Области применения режима

Несмотря на недостатки, ЕСВ имеет различные области применения, где его особенности не являются уязвимостями:

6.1 Шифрование случайных данных

Если входные данные уже являются случайным шумом (например, сессионные ключи, случайные токены), вероятность совпадения исходных блоков пренебрежимо мала, а следовательно, не возникает и проблема одинаковых шифртекстов.

6.2 Аппаратные реализации

Полная независимость блоков позволяет реализовать массовый параллелизм на аппаратном уровне. Все блоки можно шифровать одновременно, что обеспечивает максимальную пропускную способность.

6.3 Криптографические примитивы

ЕСВ используется внутри более сложных конструкций, например, в генераторах псевдослучайных чисел или в режимах, требующих обращения к блочному шифру как к идеальной псевдослучайной перестановке[2].

7 Заключение

Режим электронной кодовой книги (ЕСВ) является исторически важным и концептуально простым алгоритмом, который заложил основы для построения всех последующих режимов шифрования. Его математическая прозрачность, высокая скорость работы и возможность полного параллелизма делают его удобным инструментом в ряде специализированных задач, например, при шифровании случайных данных и в аппаратных реализациях.

Однако накопленный опыт криптоанализа и практического применения показал, что ЕСВ не способен обеспечить необходимый уровень безопасности для защиты конфиденциальной информации общего назначения. Сохранение статистических закономерностей, уязвимость к атаке по словарю и отсутствие межблочной диффузии делают этот режим устаревшим для большинства современных задач.

Список литературы

1. Dworkin M. Recommendation for Block Cipher Modes of Operation: Methods and Techniques (NIST SP 800-38A) [Электронный ресурс]. 2001. URL: <https://nvlpubs.nist.gov/nistpubs/Legacy/sp/nistspecialpublication800-38a.pdf>.
2. NIST. Decision to Revise NIST SP 800-38A, Recommendation for Block Cipher Modes of Operation: Methods and Techniques [Электронный ресурс]. 2023. URL: <https://www.nist.gov/news-events/news/2023/04/decision-revise-nist-sp-800-38a-recommendation-block-cipher-modes-operation>.
3. NIST. Block Cipher Modes Development [Электронный ресурс]. URL: <https://csrc.nist.gov/projects/block-cipher-techniques/BCM>.
4. File:Tux encrypted ecb.png [Электронный ресурс]. URL: https://commons.wikimedia.org/wiki/File:Tux_encrypted_ecb.png.
5. Valsorda F. The ECB Penguin [Электронный ресурс]. URL: <https://words.filippo.io/the-ecb-penguin/>.